SIMATS ENGINEERING

Department of Information Technology

DSA01 – Object-Oriented Programming with C++

ASSIGNMENT 1 – CO1

Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

CO1: Apply object-oriented programming principles and C++ constructs such as data types, operators and control structures to design and write structured OOP-based programs – the object-oriented paradigm and its basic principles, tokens, keywords, identifiers, constants, data types, type compatibility, variables, operators, implicit conversions, operator precedence and control structures.

Student Name	Yashwanth.G	**Register Number**	192524275
Department / Section	AIDS	**Academic Year / Semester**	2025
Faculty Name	AMRITHA	**Nature of Assignment**	Individual
Duration	1 Week	**Total Marks**	50
Date of Submission	12-11-2026	**Date of Evaluation**	
GitHub Repository	DSA0115_192524275	**Compiler**	g++ 13.3.0, -std=c++17 -Wall -Wextra

MARK SHEET (Rubric-based Evaluation)

Task	Task Title	Max	Marks Awarded	Remarks
1	Object-Oriented Analysis	4		
2	Tokens, Keywords, Identifiers and Constants	3		
3	Data Types and Compatibility	4		
4	Implicit Conversions	4		
5	Operators and Precedence	4		
6	Control Structures	4		
7	Procedural vs Object-Oriented	4		
8	Input and Output	4		
9	Implementation and Testing	4		
10	Expectation vs Execution	4		
11	Final Recommendation	4		
12A	Re-evaluate: Impact Analysis	2		
12B	Re-evaluate: Evaluation of Original Design	2		
12C	Re-evaluate: Adaptation or Redesign	2		
12D	Re-evaluate: Evidence-Based Defence	2		
TOTAL	**50**		Percentage:	

OVERALL REMARKS

Yashwanth.G		
Student Signature	**Faculty Signature**	**Program Director**

OBJECT-ORIENTED PROGRAMMING WITH C++

ASSIGNMENT 1 – CO1

Nature of Assignment: Individual | Duration: 1 Week | Total Marks: 50

Bloom's Levels Assessed: K5 – Evaluate (every task in this assignment is at K5 level)

Mandatory Execution and GitHub Submission

• Every student must individually execute every task, including all "Complete the Code" programs.

• Create a GitHub repository named DSA01-<RegisterNumber>-Assignment1 and share it with the faculty.

• Keep one folder per task (Task01 … Task12) with C++ sources, inputs and output screenshots.

• Include a README.md with name, register number, compiler, g++ command, run steps and short result notes.

• Commit progressively – at least one commit per task.

GitHub Repository Link: _____

TASK 1 – Understand the Problem and Identify the Objects (4 Marks)

Bloom's Level: K5 – Evaluate

1. Understanding the Problem

Problems in the first gate software: (i) fares off by a paisa, (ii) 10 AM and 8 PM treated inconsistently, (iii) a card went negative, (iv) fare values typed in many places. All four come from one cause: nothing owns the money and the rules. The design therefore gives each item one owner, keeps money in whole numbers, and keeps each number in one place.

Assumptions (the scenario does not state them):

A1 – Concession: students and seniors pay 50% of the base fare; the surcharge is added after the concession.

A2 – Zone: the fare depends only on the farthest zone reached.

A3 – Peak: entry time from 08:00 up to 09:59, or 17:00 up to 19:59. So 8:00 and 17:00 are peak; 10:00 and 20:00 are off-peak.

A4 – Money: stored as whole paise in a long long; percentages use integer arithmetic, rounded half-up.

A5 – Balance rule: a journey is allowed only if balance − fare ≥ 5000 paise (Rs 50). Recharge must be positive.

2. Objects, Data and Behaviour

Object	Data (attributes)	Behaviour (methods)	Why it is an object
SmartCard	balance_ (long long, paise, private)	balance(), canTravel(f), deduct(f), recharge(a)	It owns the money. Only the card may change its balance, so the Rs 50 rule can be enforced in one place.
Passenger	name, category (Regular / Student / Senior)	isConcession()	The concession depends on who travels, not on the journey.
Journey	farthest zone, hour, minute	isPeak()	One journey is one unit of work at the gate; it carries the time that decides the surcharge.
Zone	Inner, Middle, Outer (enum class)	– (a value type)	A zone is one of a fixed set of values; a type of its own stops it being mixed with ordinary ints.
FareTable	BASE_PAISE[], PEAK_SURCHARGE_PERCENT, CONCESSION_PERCENT, MIN_BALANCE_PAISE	fare(journey, concession)	All fare values and the fare formula live here, so a revision is made in one place.

3. Procedural Style versus Object-Oriented Style

In a procedural program, balance and fares are global variables and the functions are separate, so the Rs 50 rule works only if every caller remembers to call canTravel() before deduct(). In the object-oriented program the card keeps balance_ private and deduct() does the check itself. The rule moves from the caller's memory to the card's code.

4. OOP Principles in this System

Principle	Where it appears	Benefit here
Encapsulation	SmartCard hides balance_; FareTable hides the constants	No code outside the card can create a negative or unchecked balance.
Abstraction	The gate calls fare(journey) and card.deduct(f) without knowing how percentages or zones are handled	The gate code does not change when the fare formula changes.
Inheritance	StudentPassenger / SeniorPassenger can inherit from Passenger; a monthly Pass can inherit from a common FareRule base	Shared data is written once; only the concession differs.
Polymorphism	A base class DiscountRule with virtual apply(fare) overridden by ConcessionRule, WeekendRule, PeakRule	New rules (weekend discount) are added without editing the gate loop.

5. Rules Every Correct Journey Must Satisfy

No.	Rule	Where enforced
R1	After fare deducted, balance ≥ Rs 50 (5000 paise); never negative	SmartCard::deduct() / canTravel()
R2	Base fare is Rs 10 / 20 / 30 by farthest zone	FareTable::BASE_PAISE
R3	Peak is 08:00–09:59 and 17:00–19:59 only; surcharge exactly 20%	Journey::isPeak(), FareTable
R4	Students and seniors pay 50% of base (A1)	FareTable::CONCESSION_PERCENT
R5	All money in integer paise; no floating point for money	Type of every money variable
R6	Recharge must be positive; unknown zone / invalid time rejected	Input validation
R7	Each fare constant written once	constexpr in one header

6. Alternative Concept: Purely Procedural Design

```
ALGORITHM ProcessJourney_Procedural
INPUT  : zone, hour, minute            (global: balance, BASE[], MIN_BAL)
OUTPUT : gate decision, updated balance
BEGIN
  fare <- fareOf(zone, hour, minute)
  IF balance - fare >= MIN_BAL THEN
    balance <- balance - fare
    RETURN OPEN
  ELSE
    RETURN DENY
  END IF
END
```

Time: both designs do the same few steps per journey → O(1). Timed 50 million journeys: procedural 234 ms, OO 226 ms (noise). Space: sizeof(long long) = sizeof(SmartCard) = 8 bytes → O(1) per card. Changing a rule: both edit one constant, but in the procedural program every function can touch balance so all must be re-checked; in OO only SmartCard can.

TASK 2 – Tokens, Keywords, Identifiers and Constants (3 Marks)

Bloom's Level: K5 – Evaluate

1. The Five Kinds of Tokens

Token kind	Examples from the fare program	Remark

Keywords	constexpr, long, int, bool, enum, class, switch, case, if, else, return	Reserved by C++; cannot be used as names
Identifiers	Zone, BASE_PAISE, fare, canTravel, balance, MIN_BALANCE_PAISE	Names chosen by the programmer
Constants (literals)	5000, 1000, 20, 100, true, false, "Denied"	Integer, boolean, character and string literals
Operators	=, +=, *, /, %, -=, <<, ::, &&, ||, ?:, <, >=	Specify computations
Punctuators	{ } ( ) [ ] ; , :	Separate and group; carry no value

2. #define vs const vs constexpr

Feature	#define	const	constexpr
Handled by	Preprocessor	Compiler	Compiler
Has a type	No	Yes	Yes
Obeys scope	No	Yes	Yes
Compile-time value	Yes (text)	Sometimes	Always
Array size / case label	Yes	Sometimes	Yes, guaranteed

I chose constexpr because it is typed, checked at compile time and scoped. Macro trap example: #define FARE_INNER 500 + 500 then FARE_INNER*2 yields 1500, not 2000.

3. enum versus enum class

A plain enum converts to int silently. An enum class does not: names stay inside their scope (Zone::Inner) and never become int by themselves. Compiler rejects int x = Zone::Outer;. enum class can also use a small underlying type (1 byte vs 4).

4. Naming Conventions Used

• Constants: UPPER_SNAKE_CASE with unit (MIN_BALANCE_PAISE)

• Types: PascalCase (Zone, SmartCard)

• Functions / variables: camelCase (canTravel, balance)

• Private members end with _ (balance_)

5. The Single Set of Constants

```
// fare_constants.h – the ONLY place where fares and limits are written
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};   // Rs 10, 20, 30
constexpr int  PEAK_SURCHARGE_PERCENT = 20;
constexpr int  CONCESSION_PERCENT     = 50;
constexpr long long MIN_BALANCE_PAISE = 5000;            // Rs 50 must remain
constexpr int  PEAK_AM_START = 8*60,  PEAK_AM_END = 10*60;  // [08:00, 10:00)
constexpr int  PEAK_PM_START = 17*60, PEAK_PM_END = 20*60;  // [17:00, 20:00)
```

6. Alternative: #define Macros

No type, no scope, extra brackets needed. Memory: both occupy none (or O(z) for an array). Time: both O(1). The choice affects safety and maintenance, not speed.

TASK 3 – Data Types and Type Compatibility (4 Marks)

Bloom's Level: K5 – Evaluate

1. Why Money is Stored as Integer Paise

A double stores numbers in binary; values like 0.1 or 10.8 cannot be stored exactly.

```
0.1 + 0.2 = 0.30000000000000004
10.8 stored as 10.800000000000001
ten times 0.10 == 1.00 ? false
double fare 0.1*3 == 0.3 ? false
```

This is the “off by a paisa” defect. An integer such as 1080 paise is always exact.

2. Chosen Types and Sizes (g++ 13.3.0, 64-bit Linux)

Value	Type chosen	sizeof	Reason
Fares / balances	long long (paise)	8	Exact; holds network total
Zone	enum class Zone	1–4	Type-safe discrete set
Hour / minute	int	4	Small range 0–23 / 0–59
Peak / concession flags	bool	1	Two-state conditions
Journey count / totals	long long	8	3 M journeys overflow int

3. Ranges and Daily Collection

Whole-network daily total ≈ 5,626,462,900 paise. INT_MAX = 2,147,483,647 → int overflows after ≈ 1.14 million journeys (less than one day). long long holds up to 9.22×10^18.

4. Alternative: double Rupees

Time and space both O(1) / 8 bytes. Measured absolute error of double sum against exact integer-paise total grows with n; at n = 10^7 for Rs 10.80 fares the error reaches nearly two paise. long long error is always 0.

TASK 4 – Control Implicit Conversions (4 Marks)

Bloom’s Level: K5 – Evaluate

1. Predict the Output (written before running)

```
#include <iostream>
int main() {
    int a = 7, b = 2;
    double c = a / b;
    double d = static_cast<double>(a) / b;
    std::cout << c << ' ' << d << '\n';
    char ch = 'A' + 2;
    std::cout << ch << ' ' << ch + 1 << '\n';
    bool flag = 5;
    std::cout << flag << ' ' << std::boolalpha << flag << '\n';
    unsigned int u = 1;
    int s = -1;
    std::cout << (s < u ? "s < u" : "s >= u") << '\n';
}
```

My predicted output (before running):

```
3 3.5
C 68
1 true
s >= u
```

Actual output (identical):

```
3 3.5
C 68
1 true
s >= u
```

Compiler warning: comparison of integer expressions of different signedness [-Wsign-compare].

2. Explanation of Each Conversion

• a / b is integer division → 3; then stored in double. static_cast forces floating division → 3.5.
• 'A' + 2 promotes char to int, adds, then narrows back to char → 'C'. ch + 1 promotes again → 68.
• Non-zero int converts to bool true; printed as 1 then as true under boolalpha.
• s < u converts s to unsigned → large positive number > 1 → false branch "s >= u".

3. Narrowing and Brace Initialisation

int x{3.7}; and bool f{5}; are rejected at compile time with -Wnarrowing. Implicit conversion silently truncates; brace initialisation catches the defect early.

4. Alternative: Every Conversion Explicit

static_cast on built-ins produces the same machine code. Time and memory unchanged (O(1)). Errors such as narrowing are caught at compile time instead of appearing later as wrong fares.

TASK 5 – Operators and Precedence (4 Marks)

Bloom's Level: K5 – Evaluate

1. The Three Expressions of the Fare System

```
int t = hour * 60 + minute;
bool peak = (t >= 8*60 && t < 10*60) || (t >= 17*60 && t < 20*60);
fare += (fare * PEAK_SURCHARGE_PERCENT + 50) / 100;   // half-up
if (concession) fare -= (fare * CONCESSION_PERCENT + 50) / 100;
```

Precedence: * before +; comparisons before && before ||. Brackets force the intended order and improve readability.

2. Predict the Output (written before running)

Predicted:

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Actual output: identical. No warnings.

3. Operator-by-Operator Explanation

Expression	Order of application → Result
x + y * 2	* before +: 5 + (3×2) = 11
(x + y) * 2	brackets force + first: 16
x++ + ++y	x++ yields 5 (x→6); ++y yields 4 (y→4); 5+4=9
x % 4 * 2	% and * same precedence, left-to-right: (6%4)*2 = 4
-10 / 3 , -10 % 3	truncate toward zero: −3; remainder sign follows dividend: −1
for i+=3; k+=i	i = 0,3,6,9 → k = 18

4. Alternative: Lookup Table for Peak

bool peakTable[24] filled once; lookup is one array access. Both O(1). Table adds 24 bytes and duplicates the rule, so it must be kept in sync.

TASK 6 – Control Structures (4 Marks)

Bloom's Level: K5 – Evaluate

1. Choice of Structure

Decision	Structure	Reason
Base fare from zone	switch on Zone	Discrete values; -Wswitch warns if enumerator missed
Peak / concession	if / else	Range or combined tests
Allow / deny journey	if / else on canTravel()	Two outcomes with different statements
Process day's journeys	range-based for	Visits every element exactly once, no index errors

2. Pseudocode

```
ALGORITHM ComputeFare
INPUT  : z (zone), peak (boolean)
OUTPUT : fare in paise
BEGIN
  SWITCH z
    CASE Inner  : base <- BASE_PAISE[0]
    CASE Middle : base <- BASE_PAISE[1]
    CASE Outer  : base <- BASE_PAISE[2]
  END SWITCH
  IF peak THEN base <- base + base * 20 / 100 END IF
  RETURN base
END
```

3. Completed Code (TODO 1 & TODO 2)

```
long long fare(Zone z, bool peak) {
    long long base = 0;
    switch (z) {
        case Zone::Inner:  base = BASE_PAISE[0]; break;
        case Zone::Middle: base = BASE_PAISE[1]; break;
        case Zone::Outer:  base = BASE_PAISE[2]; break;
    }
    if (peak) base += base * PEAK_SURCHARGE_PERCENT / 100;
    return base;
}
bool canTravel(long long balance, long long f) {
    return balance - f >= MIN_BALANCE_PAISE;
}
```

Actual output matches expected:

```
Fare Rs 10.00 balance Rs 90
Fare Rs 36.00 balance Rs 54
Denied (low balance) balance Rs 54
```

Boundary check: 10:00 and 20:00 are off-peak; 08:00 and 17:00 are peak (half-open intervals on minutes-since-midnight).

4. Alternative: Array Lookup

Both O(1). Array uses 24 bytes; switch keeps values in instructions. Fourth zone: array needs one extra element + static_assert; switch needs a new case (compiler only warns if forgotten).

TASK 7 – Procedural vs Object-Oriented (4 Marks)

Bloom's Level: K5 – Evaluate

1. Procedural Version

Global balance + free functions. Careless deduct(9999) produces negative balance (−4599). The Rs 50 rule lives only in the caller's discipline.

2. Object-Oriented Version

```
class SmartCard {
    long long balance_;                 // private
    static constexpr long long MIN_BAL = 5000;
public:
    explicit SmartCard(long long b) : balance_(b) {}
    long long balance() const { return balance_; }
    bool canTravel(long long f) const { return balance_ - f >= MIN_BAL; }
    bool deduct(long long f) {
        if (!canTravel(f)) return false;
        balance_ -= f;
        return true;
    }
};
```

Careless deduct(9999) is refused; balance stays 5400. The rule is inside the class and the compiler prevents outside code from touching the balance.

3. Comparison

Point	Procedural	Object-oriented
Where balance lives	Global, visible to all	Private member
Who enforces Rs 50	Every caller	deduct() itself
Careless deduct(9999)	balance becomes −4599	refused; balance unchanged
Change minimum balance	One constant + review all functions	One constant inside SmartCard

4. Alternative: Public Balance

Any code can write card.balance = −500. Time and space unchanged, but R1 (minimum balance) and non-negativity can be broken by any part of the program.

TASK 8 – Safe Input and Formatted Output (4 Marks)

Bloom's Level: K5 – Evaluate

1. Recovery from Bad Input

If the operator types letters, cin enters fail state. Recovery: check ! (cin >> n), call cin.clear(), then cin.ignore(max, '\n').

2. Gate Program Highlights

• Validates zone 0–2, hour 0–23, minute 0–59.

• Skips invalid journeys with continue; processing continues.

• Money formatted with setw, setfill('0'), fixed, setprecision(2); arithmetic stays in paise.

3. Alternative: getline + stoi

Each line independent; bad line only affects itself. Time O(L) both ways. getline needs O(L) buffer space; cin >> needs none extra. For pure numeric fields cin >> is simpler; for line-oriented records getline + validation is safer.

TASK 9 – Implement and Test the Fare Logic (4 Marks)

Bloom's Level: K5 – Evaluate

1. Test Table (excerpt)

Case	Zone	Time	Expected	Result
Inner off-peak	0	12:00	1000	PASS
Outer peak	2	08:30	3600	PASS
Boundary 10:00	0	10:00	1000	PASS
Boundary 20:00	0	20:00	1000	PASS
Leaves exactly Rs 50	–	–	allowed	PASS
Leaves Rs 49.99	–	–	denied	PASS
Five-journey total	mix	mix	11000	PASS

All table cases and hand-calculated 5-journey total matched. Invalid zone / negative recharge / non-numeric hour rejected.

2. Alternative: Table-Driven Tests

One loop over a Case array. Time O(t), space O(t). Adding a new boundary is one new row of data instead of a new if-statement.

TASK 10 – Expectation vs Execution (4 Marks)

Bloom's Level: K5 – Evaluate

1. Measured Totals

Journeys	long long (exact)	double (×100)	float	int (wrapped)
20,000	37,563,600	exact	exact	fits
500,000	937,370,000	exact	exact	fits
3,000,000	5,626,462,900	exact*	−48,977 Rs error	overflow

* For these particular fares (exact binary fractions). Task 3 shows drift appears with values such as Rs 10.80.

2. Conclusions

int overflows at network scale. float loses precision above 2^24. long long stays exact and is as fast as int. Compiler warnings (-Wsign-compare, -Wsequence-point, -Wswitch) each pointed at a real defect that was fixed.

TASK 11 – Final Design Recommendation (4 Marks)

Bloom's Level: K5 – Evaluate

1. Recommended Elements (with evidence)

Element	Recommendation	Evidence
Money	long long paise everywhere	Task 3 drift; Task 10 int overflow
Constants	constexpr in one header	Task 2 macro trap
Zones	enum class Zone	Task 2 silent int conversion
Peak rule	minutes-since-midnight, half-open	Task 6/9 boundary tests
SmartCard	private balance + deduct() check	Task 7 negative-balance prevention
Compiler flags	-std=c++17 -Wall -Wextra	Task 10 warnings caught real bugs

2. Fare Revision Process

1. Edit the single line in the constants header. 2. Rebuild with -Wall -Wextra. 3. Update expected values in the table-driven test and re-run. 4. Deploy the new binary.

3. Limitations & Network Correctness

Assumptions A1–A5 must be confirmed with the authority. Concurrent updates to one card need a transaction mechanism (out of scope). The design is O(1) per journey and exact for 3 M journeys, so it remains correct for the whole network.

4. Alternative: Configuration File

O(z) start-up cost (microseconds). Avoids recompilation when fares change often, but loses compile-time checking and requires robust run-time validation of the file.

TASK 12 – Re-evaluate and Adapt the Design (8 Marks)

Bloom's Level: K5 – Evaluate

Changed requirements: fourth zone (Airport, Rs 60), 10% weekend discount (does not combine with peak surcharge), monthly pass for two chosen zones.

A. Impact Analysis (2 Marks)

Item in original design	Affected?	How
BASE_PAISE[]	Yes	Add 6000 for Airport
enum class Zone	Yes	New enumerator Airport (+ Count)
switch in fare()	Yes	Missing case → silent fare 0
fare(Zone, bool) signature	Yes	Cannot express weekday / pass / concession cleanly
SmartCard	Partly	Must honour free journeys from a pass
New: WEEKEND_DISCOUNT, Pass class	New	New constant and new concept

B. Evaluation of Original Design (2 Marks)

Worked examples showed: (1) adding Airport without a new case produced fare 0 (revenue loss); (2) weekend discount cannot be expressed with only a peak flag; (3) a pass-covered journey was refused by the Rs 50 check. The original design cannot absorb the changes without modification.

C. Adaptation – Revised Design (2 Marks)

Decision: MODIFY (keep tested core, change only what the new rules touch).

```
enum class Zone { Inner, Middle, Outer, Airport, Count };
constexpr long long BASE_PAISE[ZONE_COUNT] = {1000,2000,3000,6000};
constexpr int WEEKEND_DISCOUNT_PERCENT = 10;

struct Journey { Zone farthest; int hour, minute, weekday;
                 bool concession; int year, month; };

long long fare(const Journey& j) {
    long long f = BASE_PAISE[static_cast<int>(j.farthest)];
    if (j.concession) f -= pct(f, CONCESSION_PERCENT);
    if (isWeekend(j.weekday)) f -= pct(f, WEEKEND_DISCOUNT_PERCENT);
    else if (isPeak(j.hour, j.minute)) f += pct(f, PEAK_SURCHARGE_PERCENT);
    return f;
}

class Pass { /* two-zone mask + year/month; covers(j) */ };
class SmartCard {
    long long balance_; const Pass* pass_ = nullptr;
public:
    long long travel(const Journey& j) {
        if (pass_ && pass_->covers(j)) return 0;   // free
```

```
        long long f = fare(j);
        if (balance_ - f < MIN_BALANCE_PAISE) return -1;
        balance_ -= f; return f;
    }
};
```

D. Evidence-Based Defence (2 Marks)

All new test cases (Airport peak/off-peak, weekend discount replacing surcharge, pass-covered free journey, next-month expiry) passed. Core remains O(1) per journey; memory grows by one pointer. Keeping the tested integer-paise / encapsulation core preserved all earlier results. A full polymorphic DiscountRule hierarchy would be more extensible but is unnecessary for three small, known changes. Risks accepted: assumptions about weekend vs peak interaction and pass lifetime must be confirmed with the authority.